Inter-Process Communication (IPC)

Processes executing concurrently in the operating system may be either independent processes or cooperating processes. A process is independent if it cannot affect or be affected by the other processes executing in the system. Any process that does not share data with any other process is independent. A process is cooperating if it can affect or be affected by the other processes executing in the system. Clearly, any process that shares data with other processes is a cooperating process.

There are several reasons for providing an environment that allows process cooperation:

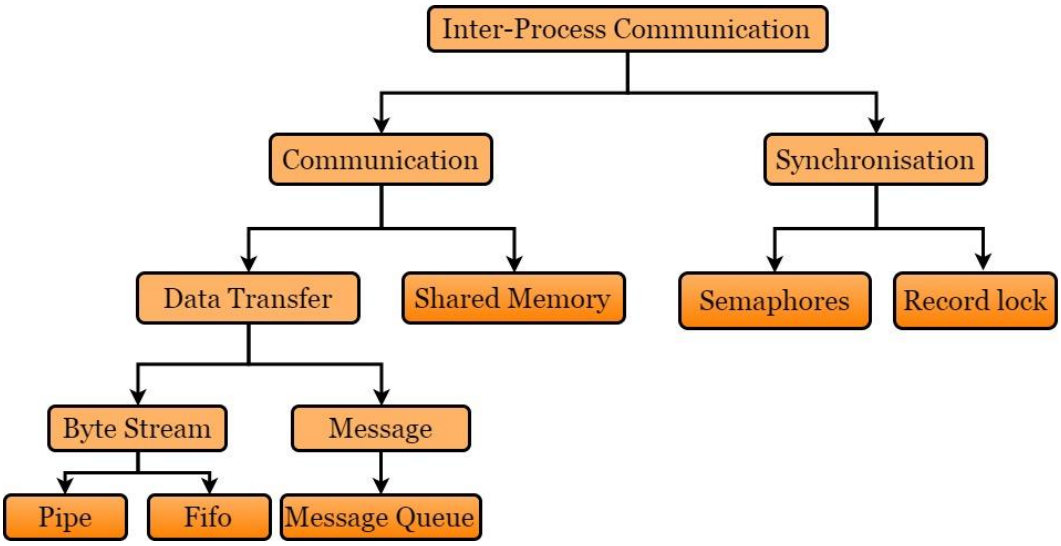
1) Information sharing: Since several users may be interested in the same piece of information (for instance, a shared file), you must provide an environment to allow concurrent access to such information.

2) Computation speedup: If you want a particular task to run faster, you must break it into subtasks, each of which will be executing in parallel with the others

3) Modularity: You may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads

4) Convenience. Even an individual user may work on many tasks at the same time. For instance, a user may be editing, printing, and compiling in parallel.

Cooperating processes require an Inter-Process Communication (IPC) mechanism that will allow them to exchange data and information.



The various communication facilities shown in diagram allow processes to exchange data with one another. There are two fundamental models of Inter-Process Communication.

1) Data Transfer: In order to communicate, one process writes data to the IPC facility, and another process reads the data. These facilities require two data transfers between user memory and kernel memory: one transfer from user memory to kernel memory during writing, and another transfer from kernel memory to user memory during reading.

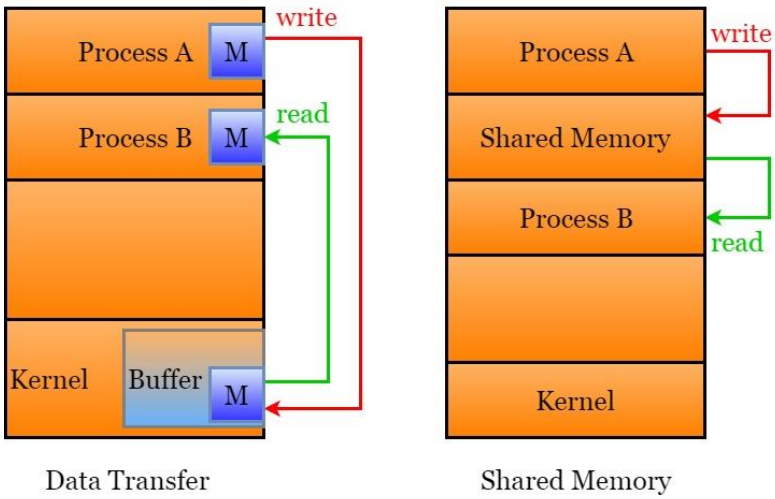
2) Shared Memory: Shared memory allows processes to exchange information by placing it in a region of memory that is shared between the processes. The kernel accomplishes this by making page-table entries in each process point to the same pages of RAM.A process can make data available to other processes by placing it in the shared memory region. Because communication doesn't require system calls or data transfer between user memory and kernel memory, shared memory can provide very fast communication.

Data transfer

You can further break data-transfer facilities into the following subcategories:

1) Byte stream: The data exchanged via pipes, FIFOs, and datagram sockets is an undelimited byte stream. Each read operation may read an arbitrary number of bytes from the IPC facility, regardless of the size of blocks written by the writer. This model mirrors the traditional UNIX "file as a sequence of bytes" model.

2) Message: The data exchanged via message queues, and datagram sockets takes the form of delimited messages. Each read operation reads a whole message, as written by the writer process. It is not possible to read part of a message, leaving the remainder on the IPC facility; nor is it possible to read multiple messages in a single read operation.



A few general features distinguish data-transfer facilities from shared memory:

Although a data-transfer facility may have multiple readers, reads are destructive. A read operation consumes data, and that data is not available to any other process.

Synchronization between the reader and writer processes is automatic. If a reader attempts to fetch data from a data-transfer facility that currently has no data, then (by default) the read operation will block until some process writes data to the facility.

Shared Memory

Note the following general points about shared memory:

Although shared memory provides fast communication, this speed advantage is offset by the need to synchronize operations on the shared memory. For example, one process should not attempt to access a data structure in the shared memory while another process is updating it. A semaphore is the usual synchronization method used with shared memory.

Data placed in shared memory is visible to all of the processes that share that memory. (This contrasts with the destructive read semantics described above for data-transfer facilities.)

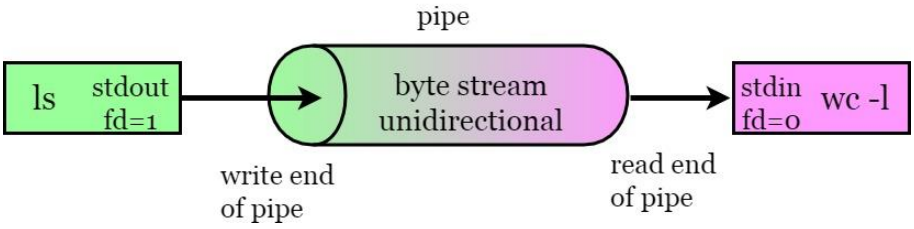
IPC Mechanism…The pipe

Pipes are the oldest method of IPC on the UNIX system, having appeared in Third Edition UNIX in the early 1970s. Pipes can be used to pass data between related processes (processes created using fork()).

Every user of the shell is familiar with the use of pipes (|) in commands such as the following, which counts the number of files in a directory:

```
$ ls | wc -l
```

In order to execute the above command, the shell creates two processes, executing ls and wc, respectively. This is done using fork() and exec(). And the data is transferred from one process to other using pipe mechanism.



In above diagram, the two processes are connected to the pipe so that the writing process (ls) has its standard output (file descriptor 1) joined to the write end of the pipe, while the reading process (wc) has its standard input (file descriptor 0) joined to the read end of the pipe. In effect, these two processes are unaware of the existence of the pipe; they just read from and write to the standard file descriptors.

Characteristics of Pipe

1) Pipes allow communication between related processes; pipes are used for communication between a parent and a child process. However, pipes can be used for communication between any two (or more) related processes, as long as the pipe was created by a common ancestor before the series of fork() calls that led to the existence of the processes. For example, a pipe could be used for communication between a process and its grandchild.

2) A pipe is a byte stream, means that there is no concept of messages or message boundaries when using a pipe. The process reading from a pipe can read blocks of data of any size, regardless of the size of blocks written by the writing process. Furthermore, the data passes through the pipe sequentially, bytes are read from a pipe in exactly the order they were written. It is not possible to randomly access the data in a pipe using lseek().

3) Pipes are unidirectional; data can travel only in one direction through a pipe. One end of the pipe is used for writing, and the other end is used for reading.

4) Pipes have a limited capacity; a pipe is simply a buffer maintained in kernel memory. This buffer has a maximum capacity. Once a pipe is full, further writes to the pipe block until the reader removes some data from the pipe. Since Linux 2.6.11, the pipe capacity is 65,536 bytes.

Creating and Using Pipe

The pipe() system call creates a new pipe.

```
#include <unistd.h>

int pipe(int pipefd[2]);

Returns 0 on success, or –1 on error.
```

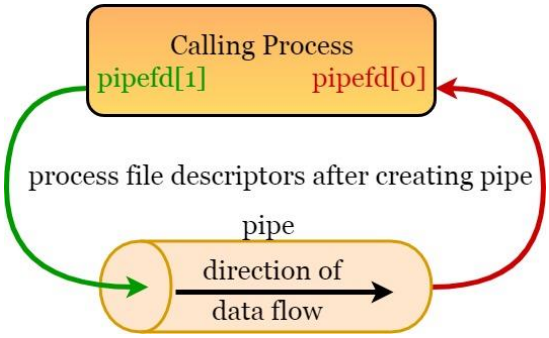
A successful call to pipe() returns two open file descriptors in the array pipefd: one for the read end of the pipe (pipefd[0]) and one for the write end (pipefd[1]).

As with any file descriptor, you can use the read() and write() system calls to perform I/O on the pipe. Once written to the write end of a pipe, data is immediately available to be read from the read end. A read() from a pipe obtains the lesser of the number of bytes requested and the number of bytes currently available in the pipe (but blocks if the pipe is empty).

```
#include<stdio.h>
#include<unistd.h>
main()
{
```

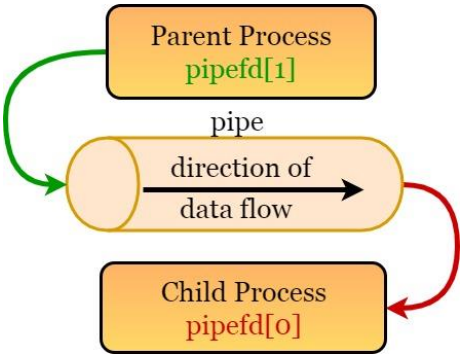
```
int p[2];
pipe(p);
printf("p[0]=%d read end\np[1]=%d write end\n",p[0],p[1]);
}

Output:
p[0]=3 read end
p[1]=4 write end
```

Following diagram shows the situation after a pipe has been created by pipe(), with the calling process having file descriptors referring to each end.



To connect two processes using a pipe, follow the pipe() call with a call to fork(). During a fork(), the child process inherits copies of its parent's file descriptors. While it is possible for the parent and child to both read from and write to the pipe, this is not usual. Therefore, immediately after the fork(), one process closes its descriptor for the write end of the pipe, and the other closes its descriptor for the read end. For example, if the parent is to send data to the child, then it would close its read descriptor for the pipe, pipefd[0], while the child would close its write descriptor for the pipe, pipefd[1]. Following code shows steps to create above situation.

```
int p[2];
pipe(p);
perror("pipe");
if(fork()==0)
{
//child process
close(p[1]);
}
else
{
//parent process
close(p[0]);
}
```



Closing unused pipe file descriptors of pipe is essential to the correct use of pipes.

The process reading from the pipe closes its write descriptor for the pipe, so that, when the other process completes its output and closes its write descriptor, the reader sees end-of-file once it has read any outstanding data in the pipe. If the reading process doesn't close the write end of the pipe, then, after the other process closes its write descriptor, the reader won't see end-of-file, even after it has read all data from the pipe. Instead, a read() would block waiting for data, because the kernel knows that there is still at least one write descriptor open for the pipe.

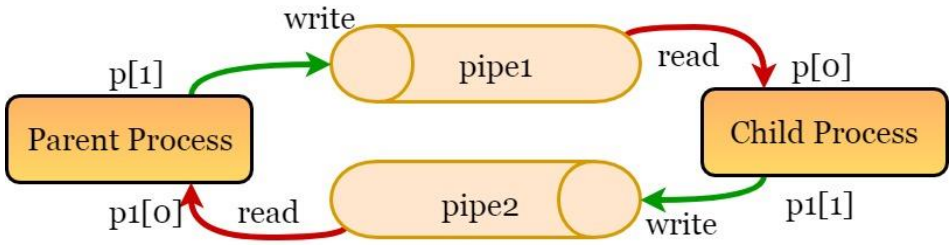
```
#include<stdio.h>
#include<unistd.h>
main()
{
int p[2],a=20;
pipe(p);
perror("pipe");

close(p[1]);                    //write end closed

a=read(p[0],&a,4);
perror("read");
printf("read returns %d\n",a);
}

Output
pipe: Success
read: Success
read returns 0
```

The writing process closes its read descriptor for the pipe for a different reason. When a process tries to write to a pipe for which no process has an open read descriptor, the kernel sends the SIGPIPE signal to the writing process. By default, this signal kills a process. A process can instead arrange to catch or ignore this signal, in which case the write() on the pipe fails with the error EPIPE (broken pipe). Receiving the SIGPIPE signal or getting the EPIPE error is a useful indication about the status of the pipe, and this is why unused read descriptors for the pipe should be closed.

```
#include<stdio.h>
#include<unistd.h>
#include<signal.h>
#include<string.h>

void myisr(int n)
{
printf("in ISR %d...%s\n",n,strsignal(n));
}
```

```
main()
{
int p[2],a=20;
pipe(p);
perror("pipe");

signal(SIGPIPE,myisr);

close(p[0]);            //read end closed

write(p[1],&a,4);
perror("write");
}

Output:
pipe: Success
in ISR 13...Broken pipe
write: Broken pipe
```

One reason that it is not usual to have both the parent and child reading from a single pipe is that if two processes try to simultaneously read from a pipe, you can't be sure which process will be the first to succeed, the two processes race for data. However, if you require bidirectional communication, there is a simpler way: just create two pipes, one for sending data in each direction between the two processes. (If employing this technique, then you need to be wary of deadlocks that may occur if both processes block while trying to read from empty pipes or while trying to write to pipes that are already full.)



Duplex Communication

One final reason for closing unused file descriptors is that it is only after all file descriptors in all processes that refer to a pipe are closed that the pipe is destroyed and its resources released for reuse by other processes. At this point, any unread data in the pipe is lost.

The following program implements communication between parent and child process. Parent process scans text from keyboard and sends to child process, which converts text from lower case to upper case and prints on screen.

```
#include<stdio.h>
```

```
#include<unistd.h>
#include<string.h>
main()
{
int p[2];

pipe(p);
perror("pipe");

if(fork()==0)
{
        //child prints on monitor
        char b[15],i;
        close(p[1]);
        while(1){
                read(p[0],b,sizeof(b));
                for(i=0;b[i];i++)
                {
                        if(b[i]>='a'&& b[i]<='z')
                        b[i]=b[i]-32;
                }
        printf("Data=%s\n",b);
        }
}
else
{
        //parent scan from keyboard
        char a[15],ch;
        close(p[0]);
        while(1)
        {
                printf("Enter data\n");
                scanf("%s",a);
                write(p[1],a,strlen(a)+1);
        }
}
}
```

Here in above program, read() blocks child process if pipe is empty. If a process attempts to read from an empty pipe, then read() will block until data is available. Similarly if a process attempts to write to a full pipe, then write blocks until sufficient space is made available by reading out from pipe. You can make non blocking I/O operations using pipe2() or fcntl() system calls.

```
#define _GNU_SOURCE
#include <fcntl.h>
#include <unistd.h>

int pipe2(int pipefd[2], int flags);
```

Flag O_NONBLOCK in pipe2() avoids read and write system calls from blocking the process.

```
while(write(p[1],&ch,1)!=-1);
```

In above statement, the write() will block till pipe gets sufficient space to write. It will never come out of while loop. But if you use non blocking flag with pipe2(), write() will fail if pipe is full and it will come out of while loop. Using this you can get the pipe size in bytes on your system. Following program prints size of pipe.

```
#include<stdio.h>
#include<unistd.h>
#include<fcntl.h>
main()
{
int p[2],c;
char ch='a';

pipe2(p,O_NONBLOCK);

c=0;

while(write(p[1],&ch,1)!=-1)
c++;

printf("size of pipe is %d\n",c);
}

Output:
size of pipe is 65536
```

Using Pipes

When a pipe is created, the file descriptors used for the two ends of the pipe are the next lowest-numbered descriptors available. Since, in normal circumstances, descriptors 0, 1, and 2 are already in use for a process, some higher-numbered descriptors will be allocated for the pipe. In the starting part of pipe discussion you have seen the ls | wc –l model, here programs that read from stdin and write to stdout are connected using a pipe, such that the standard output of one program is directed into the pipe and the standard input of the other is taken from the pipe. How’s this possible?

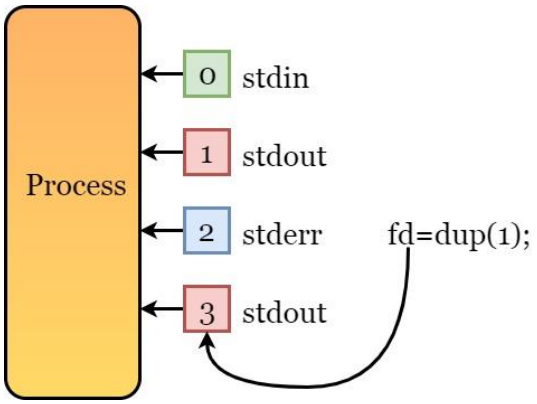
This situation can be achieved by duplicating file descriptors. This can be achieved with system calls dup() and dup2().

```
#include <unistd.h>

int dup(int oldfd);

int dup2(int oldfd, int newfd);
```

The dup() uses the lowest-numbered unused descriptor for the new descriptor and dup2() makes newfd be the copy of oldfd, closing newfd first if necessary.

The following program shows how dup() works,

```
main()
{
int fd;
fd=dup(1);
printf("fd=%d\n",fd);
write(fd, "Hello",5);
}
Output:
fd=3
Hello
```



The diagram explains well; you duplicated file descriptor 1 which is stdout. The dup() assigns lowest unused descriptor, here 3. So file descriptor 1 and 3 both are now pointing to stdout. Thus write() prints Hello on screen i.e. standard output device.

But to achieve "ls | wc –l", you have to build a pipe. After building a pipe, create two child processes. The first child binds its standard output to the write end of the pipe and then execs ls. The second child binds its standard input to the read end of the pipe and then execs wc. Binding standard output to write end of file can be achieved as shown below,

```
int pfd[2];

pipe(pfd);              //Allocates file descriptors 3 and 4 for pipe

        /* Other steps here, e.g., fork() */

close(1);               //Free file descriptor 1

dup(pfd[1]);  //Duplication uses lowest free file descriptor, i.e., fd 1 */
```

The end result of the above steps is that the process's standard output is bound to the write end of the pipe. A corresponding set of calls can be used to bind a process's standard input to the read

end of the pipe. Note that these steps depend on the assumptions that file descriptors 0, 1, and 2 for a process are already open. The shell normally ensures this for each program it executes.

In above code close() is followed by dup(). Instead you can write single call that is dup2() which makes new file descriptor the copy of old file descriptor by closing new file descriptor if necessary.

```
int pfd[2];

pipe(pfd);              //Allocates file descriptors 3 and 4 for pipe

        /* Other steps here, e.g., fork() */

dup2(pfd[0],0);     //Frese fd 0 and Duplicates pfd[0] to 0 */
```

The end result of the above steps is that the process's standard input is bound to the read end of the pipe.

The following program implements " ls | wc -l " command.

```
#include<stdio.h>
#include<unistd.h>
main()
{
int p[2];
pipe(p);

if(fork()==0)
{
//child1 executes ls
        close(p[0]);

        close(1);

        dup(p[1]);

        execlp("ls","ls",(char*)NULL);
}
else
{
        if(fork()==0)
        {
        //child2 executes wc -l
                close(p[1]);

                dup2(p[0],0);
```

```
                execlp("wc","wc","-l",(char*)NULL);

        }

        else
        {
        //parent closes unused descriptors
                close(p[0]);
                close(p[1]);

                wait(0);
                wait(0);
}
}
}
```

You can write the similar code to implement “ps –el | grep pts/0”.

Pipe is very first inter process communication mechanism which has main drawback that using pipe only related processes can communicate. Other mechanism for byte stream data transfer type IPC is named pipe or FIFO.

IPC Mechanism…The FIFO

A FIFO is similar to a pipe; the principal difference is that a FIFO has a name within the file system and is opened in the same way as a regular file. Thus FIFO is also called as named pipe. This allows a FIFO to be used for communication between unrelated processes. A FIFO has a write end and a read end, and data is read from the pipe in the same order as it is written. This fact gives FIFOs their name: first in, first out. Once a FIFO has been opened, you use the same I/O system calls as are used with other files (open(), read(), write() and close()). When all descriptors referring to a FIFO have been closed, any outstanding data is discarded.

You can create a FIFO from the shell using the mkfifo command:

```
$ mkfifo f1
$ ls -l f1
prw-rw-r-- 1 amruta amruta 0 Aug 3 15:02 f1
```

The f1 is the name of the FIFO to be created. When listed with ls –l, a FIFO is shown with the type p in the first column. When applied to a FIFO (or pipe), fstat() and stat() return a file type of S_IFIFO in the st_mode field of the stat structure.

The mkfifo() function creates a new FIFO with the given pathname.

```
#include<sys/types.h>
```

```
#include <sys/stat.h>

int mkfifo(const char *pathname, mode_t mode);

Returns 0 on success, or –1 on error
```

The mkfifo() makes a FIFO special file with name pathname. The mode specifies the FIFO's permissions. It is modified by the process's umask in the usual way: the permissions of the created file are (mode & ~umask).

A FIFO special file (a named pipe) is similar to a pipe, except that it is accessed as part of the file system. It can be opened by multiple processes for reading or writing. When processes are exchanging data via the FIFO, the kernel passes all data internally without writing it to the file system. Thus, the FIFO special file has no contents on the file system; the file system entry merely serves as a reference point so that processes can access the pipe using a name in the file system.

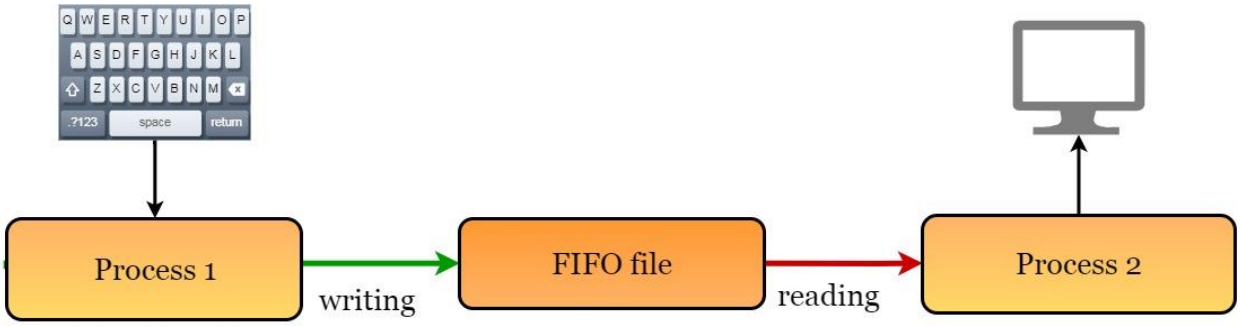
The kernel maintains exactly one pipe object for each FIFO special file that is opened by at least one process. The FIFO must be opened on both ends (reading and writing) before data can be passed. Once you have created a FIFO special file in this way, any process can open it for reading or writing, in the same way as an ordinary file. However, it has to be open at both ends simultaneously before you can proceed to do any input or output operations on it. Opening a FIFO for reading normally blocks until some other process opens the same FIFO for writing, and vice versa. If the opposite end of a FIFO is already open (perhaps because a pair of processes have already opened each end of the FIFO), then open() succeeds immediately. When a process tries to write to a FIFO that is not opened for read on the other side, the process is sent a SIGPIPE signal.

Simplex Communication using FIFO

simplexfifo1.c	simplexfifo2.c

#include<stdio.h>
#include<unistd.h>
#include<sys/stat.h>
#include<sys/types.h>
 	
main()
{
int fd;
char a[15],ch;
 	
mkfifo("fifofile",0666);
	#include<stdio.h>
	#include<unistd.h>
	#include<sys/types.h>
	#include<sys/stat.h>
	
	main()
	{
	int fd;
	char b[15];

perror("mkfifo"); *fd=open("fifofile",O_WRONLY);* *perror("open");* *do* *{* *printf("Process 1\n");* *printf("Enter data\n");* *scanf("%s",a);* *write(fd,a,strlen(a)+1);* *perror("write");* *}while(1);* *}* *$ cc simplexfifo1.c -o sf1* *Output:* *./sf1* *mkfifo: Success* *open: Success* *Process 1* *Enter data* *Vector* *Process 1* *Enter data* *India*	*fd=open("fifofile",O_RDONLY);* *perror("open");* *printf("process 2\n");* *while(1)* *{* *read(fd,b,sizeof(b));* *printf("%s\n",b);* *}* *}* *$ cc simplexfifo2.c -o sf2* *Output:* *$ ./sf2* *open: Success* *process 2* *Vector* *India*

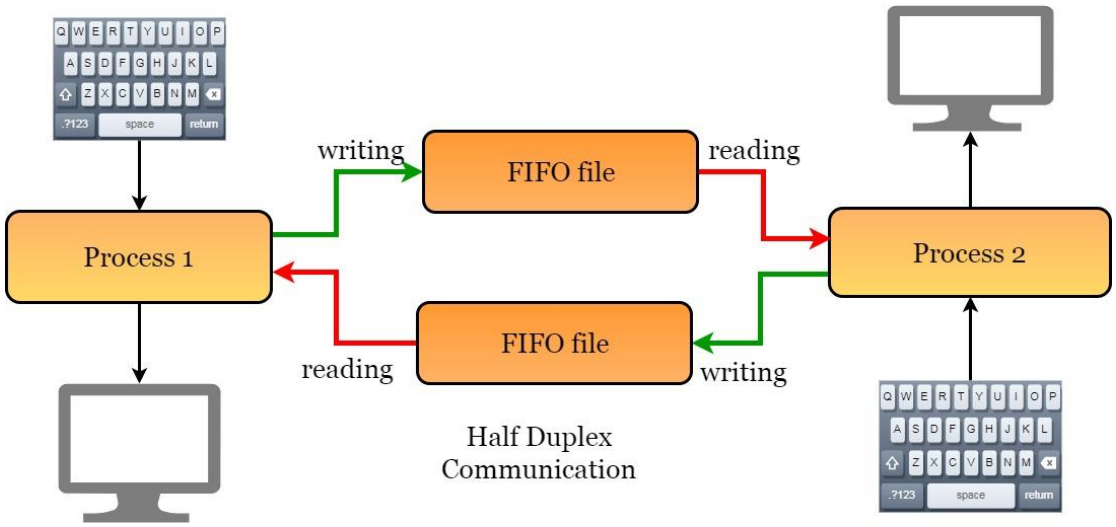


Simplex
Communication

Half Duplex Communication using FIFO

```
main()
{
int fdr,fdw;
char a[100],b[100];

mkfifo("fifohd1",0600);
perror("mkfifo");
mkfifo("fifohd2",0600);
perror("mkfifo");

fdr=open("fifohd1",O_RDWR);
fdw=open("fifohd2",O_RDWR);
while(1)
{
printf("You...\n");
scanf(" %[^\n]",a);

write(fdw,a,strlen(a)+1);

bzero(b,sizeof(b));
read(fdr,b,sizeof(b));

printf("\t\tFrom                    Hyderabad
...\n\t\t%s\n\t\t******\n",b);
}
}
$ ./hdf1
mkfifo: Success
mkfifo: Success
You ...
Hi,
                From Hyderabad...
                Hello
                ******
You...
How are you?
```

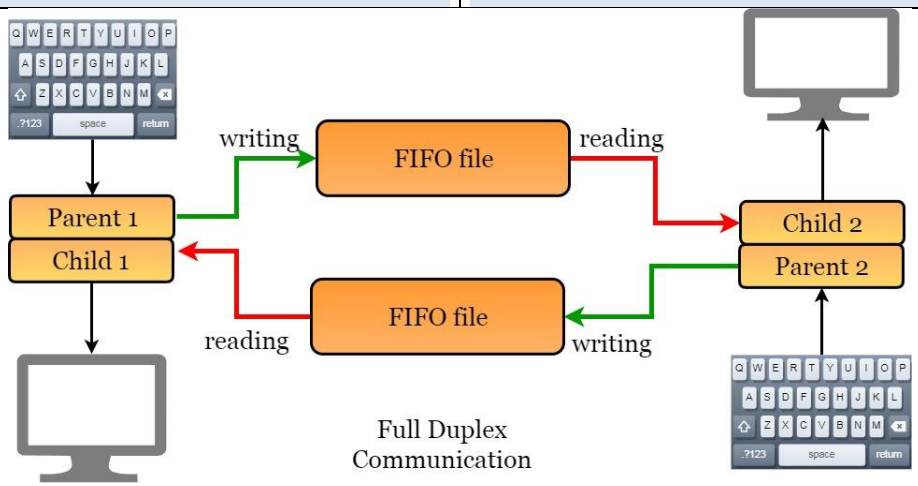
```
main()
{
int fdr,fdw;
char a[100],b[100];

fdr=open("fifohd2",O_RDWR);
fdw=open("fifohd1",O_RDWR);
while(1)
{
bzero(b,sizeof(b));
read(fdr,b,sizeof(b));

printf("\t\tFrom                    Bangalore
...\n\t\t%s\n\t\t******\n",b);

printf("You ...\n");
scanf(" %[^\n]",a);

write(fdw,a,strlen(a)+1);
}
}

$ ./hdf2
Hello
                From Bangalore...
                Hi,
                ******
You...
Whats up?
                From Bangalore...
                How are you?
                ******
```



Full Duplex Communication using FIFO

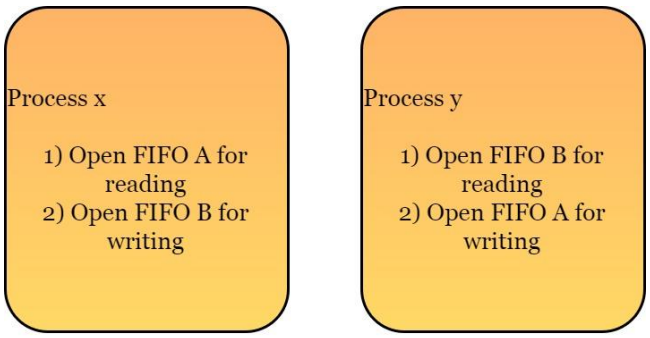
#include<stdio.h> *#include<unistd.h>* *#include<sys/types.h>* *#include<sys/stat.h>* *#include<string.h>* *#include<fcntl.h>* *main()* *{* *int fd1,fd2;* *char a[20];* *fd1=open("f1",O_RDWR);* *perror("open_1");* *fd2=open("f2",O_RDWR);* *perror("open_2");* *if(fork()==0)* *{* *//child reading fifo 2* *while(1)* *{* *read(fd2,a,sizeof(a));* *printf("\t%s\n",a);* *}* *}	*#include<stdio.h>* *#include<unistd.h>* *#include<sys/types.h>* *#include<sys/stat.h>* *#include<string.h>* *#include<fcntl.h>* *main()* *{* *int fd1,fd2;* *char a[20];* *mkfifo("f1",0666);* *mkfifo("f2",0666);* *fd1=open("f1",O_RDWR);* *perror("open_1");* *fd2=open("f2",O_RDWR);* *perror("open_2");* *if(fork()==0)* *{* *//child reading fifo 2* *while(1)* *{* *read(fd1,a,sizeof(a));* *printf("\t%s\n",a);* *}* *}

else *{* *//parent writing fifo 1* *while(1)* *{* *scanf("%s",a);* *write(fd1,a,strlen(a)+1);* *}* *}* *}*	*else* *{* *//parent writing fifo 1* *while(1)* *{* *scanf("%s",a);* *write(fd2,a,strlen(a)+1);* *}* *}* *}*



Dead Lock

A deadlock is a situation where two or more process are blocked because each is waiting on the other process(es) to complete some action. The two processes shown in below figure are deadlocked. Each process is blocked waiting to open a FIFO for reading. This blocking would not happen if each process could perform its second step (opening the other FIFO for writing).



Dead Lock between process X and Y opening two FIFOs

This particular deadlock problem could be solved by reversing the order of steps 1 and 2 in process Y, while leaving the order in process X unchanged, or vice versa. This is what you can see in half duplex communication, thus avoiding dead lock.

However, such an arrangement of steps may not be easy to achieve in some applications. Instead, you can resolve the problem by having either process, or both, specify the O_NONBLOCK flag when opening the FIFOs for reading.

Limitations of FIFO

1) When process writing data into FIFO, reading process must and should be available.

2) If multiple processes are reading from FIFO, which process will get the data is purely depends on race condition. In named pipe there is no such mechanism where transmitter can decide which receiver will get data, i.e. no addressing mechanism.

These limitations of FIFO are taken care in the next IPC mechanism i.e. message queue. Message queues can be used to pass messages between processes.

IPC Mechanism…The Message Queue

Message queues are somewhat like pipes, but differ in two important respects. First, message boundaries are preserved, so that readers and writers communicate in units of messages, rather than via an undelimited byte stream. Second, each message includes an integer type field, and it is possible to select messages by type, rather than reading them in the order in which they were written.

Message queues allow processes to exchange data in the form of messages. Although message queues are similar to pipes and FIFOs in some respects, they also differ in important ways:

1) The handle used to refer to a message queue is the identifier returned by a call to msgget(). These identifiers are not the same as the file descriptors used for most other forms of I/O on UNIX systems.

2) Communication via message queues is message-oriented; that is, the reader receives whole messages, as written by the writer. It is not possible to read part of a message, leaving the remainder in the queue, or to read multiple messages at a time. This contrasts with pipes, which provide an undifferentiated stream of bytes (i.e., with pipes, the reader can read an arbitrary number of bytes at a time, irrespective of the size of data blocks written by the writer).

3) In addition to containing data, each message has an integer type. Messages can be retrieved from a queue in first-in, first-out order or retrieved by type.

Creating or Opening a Message Queue

The msgget() system call creates a new message queue or obtains the identifier of an existing queue.

```
#include <sys/types.h> /* For portability */

#include <sys/msg.h>

int msgget(key_t key, int msgflg);

Returns message queue identifier on success, or –1 on error
```

The key argument is a key generated using one of the methods described below. The msgflg argument is a bit mask that specifies the permissions to be placed on a new message queue or checked against an existing queue. Zero or more of the following flags can be ORed (|) in msgflg to control the operation of msgget():

IPC_CREAT: If no message queue with the specified key exists, create a new queue.

IPC_EXCL: If IPC_CREAT was also specified, and a queue with the specified key already exists, fail with the error EEXIST.

IPC Key: Keys are integer values represented using the data type key_t. The msgget() call translates a key into the corresponding integer IPC identifier. These calls guarantee that if we create a new IPC object, then that object will have a unique identifier, and that if we specify the key of an existing object, then we'll always obtain the (same) identifier for that object. Internally, the kernel maintains data structures mapping keys to identifiers for each IPC mechanism. There are three possibilities to provide unique key:

1) Randomly choose some integer key value, which is typically placed in a header file included by all programs using the IPC object.

2) Specify the IPC_PRIVATE constant as the key value to the get call when creating the IPC object, which always results in the creation of a new IPC object that is guaranteed to have a unique key.

3) Employ the ftok() function to generate a (likely unique) key.

The msgget() system call begins by searching the set of all existing message queues for one with the specified key. If a matching queue is found, the identifier of that queue is returned (unless both IPC_CREAT and IPC_EXCL were specified in msgflg, in which case an error is returned). If no matching queue was found and IPC_CREAT was specified in msgflg, a new queue is created and its identifier is returned.

The following program provides a command-line interface to the msgget() system call. The program permits the use of command-line options and arguments to specify the key number.

```
#include<stdio.h>
#include<stdarg.h>
```

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>

main(int argc, char *argv[])
{
int id;

if(argc!=2)
{
printf("Usage: ./a.out keyno\n");
return;
}

id=msgget(atoi(argv[1]),IPC_CREAT|0666);

if(id<0)
{
perror("msgget");
return;
}

printf("id=%d\n",id);
}
```

The ipcs and ipcrm Commands

The ipcs and ipcrm commands are the System V IPC analogs of the ls and rm file commands. Using ipcs, you can obtain information about IPC objects message queue, shared memory and semaphores on the system. By default, ipcs displays all objects. You can give options to ipcs command as -q, -m and -s for message queue, shared memory and semaphore respectively.

```
$ ipcs

------ Shared Memory Segments --------

key                 shmid     owner     perms     bytes     nattch    status

0x00000000 1343488   amruta    600       524288    2         dest

------ Semaphore Arrays --------

key       semid     owner     perms     nsems

------ Message Queues --------
```

```
key      msqid     owner     perms     used-bytes   messages
```

By default, for each object, ipcs displays the key, the identifier, the owner, and the permissions (expressed as an octal number), followed by information specific to the object

Here You can see no entry is shown for message queue and semaphores. But if you run above program with key value as 4, you can see the message queue created.

```
$./a.out   4

$ipcs  - q       #displays only message queues

------ Message Queues --------

key                     msqid     owner     perms     used-bytes   messages

0x00000004          0        amruta           666            0                      0
```

The ipcrm command deletes an IPC object. The general form of this command is one of the following

```
$ ipcrm -X key

$ ipcrm -x id
```

In the above, we either specify key as an IPC object key or id as an IPC object identifier, and the letter x is replaced by an uppercase or lowercase q (for message queues), s (for semaphores), or m (for shared memory). Thus you can use following command to delete message queue with key number 4.

```
$ipcrm  - Q 4
```

Exchanging Messages

The msgsnd() and msgrcv() system calls perform I/O on message queues. The first argument to both system calls (msqid) is a message queue identifier. The second argument, msgp, is a pointer to a programmer-defined structure used to hold the message being sent or received. This structure has the following general form:

```
struct mymsg {

long mtype; /* Message type */

char mtext[]; /* Message body */

}
```

A zero-length mtext field is permitted, and is sometimes useful if the information to be conveyed can be encoded solely in the message type or if the existence of a message is in itself sufficient information for the receiving process.

Sending Messages

The msgsnd() system call writes a message to a message queue.

```
#include <sys/types.h>

#include <sys/msg.h>

int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);

Returns 0 on success, or –1 on error
```

To send a message with msgsnd(), we must set the mtype field of the message structure to a value greater than 0 and copy the desired information into the programmer-defined mtext field. The msgsz argument specifies the number of bytes contained in the mtext field. The final argument, msgflg, is a bit mask of flags controlling the operation of msgsnd(). Only one such flag is defined:

IPC_NOWAIT: Perform a nonblocking send. Normally, if a message queue is full, msgsnd() blocks until enough space has become available to allow the message to be placed on the queue. However, if this flag is specified, then msgsnd() returns immediately with the error EAGAIN.

Writing a message to a message queue requires write permission on the queue. Following program shows how to send message to message queue.

```
#include<stdio.h>
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>
#include<string.h>

struct msgbuf
{
long mtype;
char data[20];
};

main(int argc,char **argv)
{
if(argc!=3)
```

```
{
printf("Usage:./snd msgtype message\n");
return;
}

struct msgbuf v;
v.mtype=atoi(argv[1]);
strcpy(v.data,argv[2]);

int id;
id=msgget(1,IPC_CREAT|0660);
if(id<0)
{
perror("msgget");
return;
}

msgsnd(id,&v,strlen(v.data)+1,0);
perror("msgsnd");
}
```

Output:

```
$ ./a.out 2 Vector

msgsnd: Success

$ ./a.out 2 India

msgsnd: Success

$ ./a.out 3 Bangalore

msgsnd: Success

$ ipcs -q

------ Message Queues --------

key        msqid    owner    perms    used-bytes   messages

0x00000001 65538    amruta   666      23           3
```

Receiving Messages

The msgrcv() system call reads (and removes) a message from a message queue, and copies its contents into the buffer pointed to by msgp.

```
#include <sys/types.h>

#include <sys/msg.h>

ssize_t msgrcv(int msqid, void *msgp, size_t maxmsgsz, long msgtyp, int msgflg);

Returns number of bytes copied into mtext field, or –1 on error
```

The maximum space available in the mtext field of the msgp buffer is specified by the argument maxmsgsz. If the body of the message to be removed from the queue exceeds maxmsgsz bytes, then no message is removed from the queue, and msgrcv() fails with the error E2BIG.

Queue position	Message Type	Message body
1	3	Vector
2	2	India
3	3	Bangalore
4	1	Hyderabad
5	2	Chennai
6	4	Pune

Messages need not be read in the order in which they were sent. Instead, you can select messages according to the value in the mtype field. This selection is controlled by the msgtyp argument, as follows:

1) If msgtyp equals 0, the first message from the queue is removed and returned to the calling process.

   ```
   msgrcv(--, --, -- 0,--);
   ```

   This will read the first message “vector”(1) and then receive messages in the sequential order.

2) If msgtyp is greater than 0, the first message in the queue whose mtype equals msgtyp is removed and returned to the calling process. By specifying different values for msgtyp, multiple processes can read from a message queue without racing to read the same

messages. One useful technique is to have each process select messages matching its process ID.

```
msgrcv(--, --, -- ,2,--);
```

This will read first message of type 2, i.e. “India”(2) and then “Chennai”(2).

3) If msgtyp is less than 0, treat the waiting messages as a priority queue. The first message of the lowest mtype less than or equal to the absolute value of msgtyp is removed and returned to the calling process.

```
msgrcv(--, --, -- ,-3,--);
```

This can read messages of types 1, 2 and 3. It will start receiving from lowest message type, i.e. “Hyderabad”(1) then “India”(2), “Chennai”(2), “Vector”(3) and at last “Bangalore” (3)

The msgflg argument is a bit mask formed by ORing together zero or more of the following flags:

IPC_NOWAIT: Perform a nonblocking receive. Normally, if no message matching msgtyp is in the queue, msgrcv() blocks until such a message becomes available. Specifying the IPC_NOWAIT flag causes msgrcv() to instead return immediately with the error ENOMSG.

MSG_EXCEPT: This flag has an effect only if msgtyp is greater than 0, in which case it forces the complement of the usual operation; that is, the first message from the queue whose mtype is not equal to msgtyp is removed from the queue and returned to the caller. This flag is Linux-specific, and is made available from <sys/msg.h> only if _GNU_SOURCE is defined.

```
msgrcv(--, --, -- ,3, MSG_EXCEPT);
```

This will receive first message in the queue not equal to 3. Here it will receive “India”(2).

MSG_NOERROR: By default, if the size of the mtext field of the message exceeds the space available (as defined by the maxmsgsz argument), msgrcv() fails. If the MSG_NOERROR flag is specified, then msgrcv() instead removes the message from the queue, truncates its mtext field to maxmsgsz bytes, and returns it to the caller. The truncated data is lost.

Reading a message from a message queue requires read permission on the queue.

```
#include<stdio.h>
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>
#include<string.h>
```

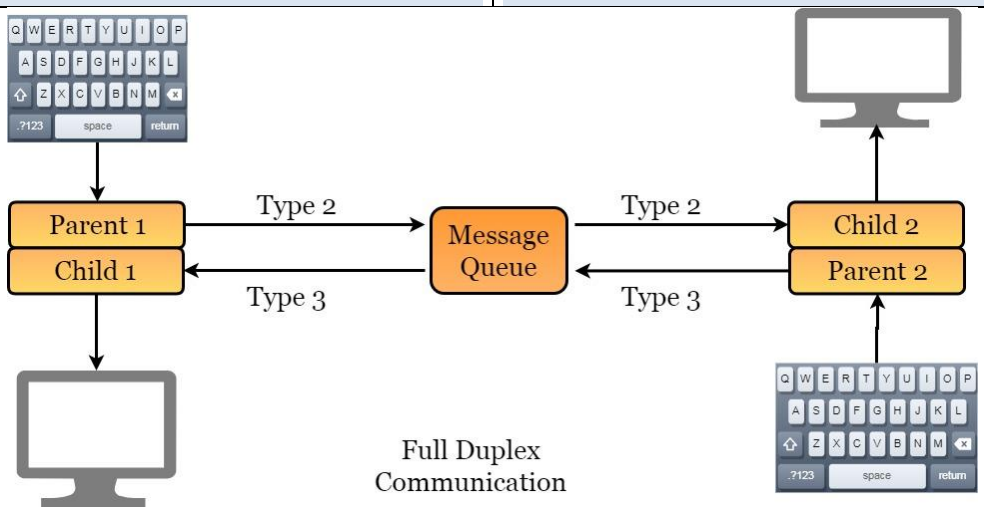
```
struct msgbuf
{
long mtype;
char data[20];
};

main(int argc,char **argv)
{

if(argc!=2)
{
printf("Usage:./rcv msgtype\n");
return;
}

struct msgbuf v;

int id;
id=msgget(1,IPC_CREAT|0660);
if(id<0)
{
perror("msgget");
return;
}

msgrcv(id,&v,sizeof(v.data),atoi(argv[1]),0);

perror("msgrcv");
printf("data:%s\n",v.data);
}
```

Full Duplex Communication using message queue

#include<stdio.h> #include<sys/types.h> #include<sys/ipc.h> #include<sys/msg.h> #include<unistd.h> #include<string.h> struct msgbuf {	#include<stdio.h> #include<sys/types.h> #include<sys/ipc.h> #include<sys/msg.h> #include<unistd.h> #include<string.h> struct msgbuf {

```
long mtype;
char data[25];
};
main()
{
int id;
struct msgbuf v;

id=msgget(2,IPC_CREAT|0666);

if(fork()==0)
{
while(1){
msgrcv(id,&v,sizeof(v.data),3,0);
printf("\t%s\n",v.data);
}}
else
{
while(1){
scanf("%s",v.data);
v.mtype=2;
msgsnd(id,&v,strlen(v.data)+1,0);
}}
}
```

```
long mtype;
char data[25];
};
main()
{
int id;
struct msgbuf v;

id=msgget(2,IPC_CREAT|0666);

if(fork()==0)
{
while(1){
msgrcv(id,&v,sizeof(v.data),2,0);
printf("\t%s\n",v.data);
}}
else
{
while(1){
scanf("%s",v.data);
v.mtype=3;
msgsnd(id,&v,strlen(v.data)+1,0);
}}
}
```



Message Queue Control Operations

The msgctl() system call performs control operations on the message queue identified by msqid.

```
#include <sys/types.h>
```

```
#include <sys/msg.h>

int msgctl(int msqid, int cmd, struct msqid_ds *buf);

Returns 0 on success, or –1 on error
```

The cmd argument specifies the operation to be performed on the queue. It can be one of the following:

IPC_RMID: Immediately remove the message queue object and its associated msqid_ds data structure. All messages remaining in the queue are lost, and any blocked reader or writer processes are immediately awakened, with msgsnd() or msgrcv() failing with the error EIDRM. The third argument to msgctl() is ignored for this operation.

IPC_STAT: Place a copy of the msqid_ds data structure associated with this message queue in the buffer pointed to by buf.

IPC_SET: Update selected fields of the msqid_ds data structure associated with this message queue using values provided in the buffer pointed to by buf.

Message Queue Associated Data Structure

Each message queue has an associated msqid_ds data structure of the following form:

```
struct msqid_ds
{
struct ipc_perm msg_perm;          /* Ownership and permissions */
time_t msg_stime;                  /* Time of last msgsnd() */
time_t msg_rtime;                  /* Time of last msgrcv() */
time_t msg_ctime;                  /* Time of last change */
unsigned long __msg_cbytes;        /* Number of bytes in queue */
msgqnum_t msg_qnum;                /* Number of messages in queue */
msglen_t msg_qbytes;               /* Maximum bytes in queue */
pid_t msg_lspid;                   /* PID of last msgsnd() */
pid_t msg_lrpid;                   /* PID of last msgrcv() */
};
```

Disadvantages of Message Queue

A distinctive feature of System V message queues is the ability to attach a numeric type to each message. This provides for two possibilities that may be useful to applications: reading processes may select messages by type, or they may employ a priority-queue strategy so that higher-priority messages (i.e., those with lower message type values) are read first. However, System V message queues have a number of disadvantages.

1) Message queues are referred to by identifiers, rather than the file descriptors used by most other UNIX I/O mechanisms. This means that a variety of file descriptor–based I/O techniques can't be applied to message queues. Furthermore, writing programs that simultaneously handle inputs from both message queues and file descriptor–based I/O mechanisms requires code that is more complex than code that deals with file descriptors alone.

2) The use of keys, rather than filenames, to identify message queues results in additional programming complexity and also requires the use of ipcs and ipcrm instead of ls and rm.

3) Message queues are connectionless, and the kernel doesn't maintain a count of the number of processes referring to the queue as is done with pipes, FIFOs, and sockets.

4) Message queue is slowest IPC mechanism.

IPC Mechanism…The Shared Memory

Shared memory allows two or more processes to share the same region (usually referred to as a segment) of physical memory. Since a shared memory segment becomes part of a process's user-space memory, no kernel intervention is required for IPC. All that is required is that one process copies data into the shared memory; that data is immediately available to all other processes sharing the same segment. This provides fast IPC by comparison with techniques such as pipes or message queues, where the sending process copies data from a buffer in user space into kernel memory and the receiving process copies in the reverse direction. Each process also incurs the overhead of a system call to perform the copy operation.

On the other hand, the fact that IPC using shared memory is not mediated by the kernel means that, typically, some method of synchronization is required so that processes don't simultaneously access the shared memory (e.g., two processes performing simultaneous updates, or one process fetching data from the shared memory while another process is in the middle of updating it). System V semaphores are a natural method for such synchronization. Other method is file locks.

Steps for Using Shared Memory

In order to use a shared memory segment, following steps are to be performed:

1) Call shmget() to create a new shared memory segment or obtain the identifier of an existing segment (i.e., one created by another process). This call returns a shared memory identifier for use in later calls.

2) Use shmat() to attach the shared memory segment; that is, make the segment part of the virtual memory of the calling process.

3) At this point, the shared memory segment can be treated just like any other memory available to the program. In order to refer to the shared memory, the program uses the addr value returned by the shmat() call, which is a pointer to the start of the shared memory segment in the process's virtual address space.

4) Call shmdt() to detach the shared memory segment. After this call, the process can no longer refer to the shared memory. This step is optional, and happens automatically on process termination.

5) Call shmctl() to delete the shared memory segment. The segment will be destroyed only after all currently attached processes have detached it. Only one process needs to perform this step.

Creating a Shared Memory Segment

The shmget() system call creates a new shared memory segment or obtains the identifier of an existing segment. The contents of a newly created shared memory segment are initialized to 0.

```
#include <sys/types.h>

#include <sys/shm.h>

int shmget(key_t key, size_t size, int shmflg);

Returns shared memory segment identifier on success, or –1 on error
```

When you use shmget() to create a new shared memory segment, size specifies a positive integer that indicates the desired size of the segment, in bytes. The kernel allocates shared memory in multiples of the system page size, so size is effectively rounded up to the next multiple of the system page size. If you are using shmget() to obtain the identifier of an existing segment, then size has no effect on the segment, but it must be less than or equal to the size of the segment. The shmflg argument performs the same task as for the other IPC get calls, specifying the permissions to be placed on a new shared memory segment or checked against an existing segment. In addition, zero or more of the following flags can be ORed (|) in shmflg to control the operation of shmget():

IPC_CREAT: If no segment with the specified key exists, create a new segment.

IPC_EXCL: If IPC_CREAT was also specified, and a segment with the specified key already exists, fail with the error EEXIST.

On success, shmget() returns the identifier for the new or existing shared memory segment.

```
#include<stdio.h>
#include<sys/ipc.h>
```

```
#include<sys/shm.h>

main(int argc,char **argv)

{

int id;
char *p;

id=shmget(atoi(argv[1]),atoi(argv[2]),IPC_CREAT|0666);
perror("shmget");

}

$ ./a.out 4 10

shmget: Success

p$ ipcs -m
------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x00000004     1638413     amruta     666     10     0

$ ./a.out 4 30
shmget: Invalid argument

$ ./a.out 4 2
shmget: Success
```

Using Shared Memory

Without attaching shared memory to process, process can not access it. The shmat() system call attaches the shared memory segment identified by shmid to the calling process's virtual address space. When a process no longer needs to access a shared memory segment, it can call shmdt() to detach the segment from its virtual address space.

```
#include <sys/types.h>

#include <sys/shm.h>

void *shmat(int shmid, const void *shmaddr, int shmflg);

Returns address at which shared memory is attached on success,

or (void *) –1 on error

int shmdt(const void *shmaddr);
```

Returns 0 on success, or –1 on error

The shmaddr argument and the setting of the SHM_RND bit in the shmflg bit-mask argument control how the segment is attached:

1) If shmaddr is NULL, then the segment is attached at a suitable address selected by the kernel. This is the preferred method of attaching a segment.

2) If shmaddr is not NULL, and SHM_RND is not set, then the segment is attached at the address specified by shmaddr, which must be a multiple of the system page size.

3) If shmaddr is not NULL, and SHM_RND is set, then the segment is mapped at the address provided in shmaddr, rounded down to the nearest multiple of the constant SHMLBA (shared memory low boundary address).

Specifying a non-NULL value for shmaddr (i.e., either the second or third option listed above) is not recommended, for the following reasons:

1) It reduces the portability of an application. An address valid on one UNIX implementation may be invalid on another.

2) An attempt to attach a shared memory segment at a particular address will fail if that address is already in use.

As its function result, shmat() returns the address at which the shared memory segment is attached. This value can be treated like a normal C pointer; the segment looks just like any other part of the process's virtual memory. Typically, you can assign the return value from shmat() to a pointer to some programmer-defined structure, in order to impose that structure on the segment.

To attach a shared memory segment for read-only access, we specify the flag SHM_RDONLY in shmflg. Attempts to update the contents of a read-only segment result in a segmentation fault (the SIGSEGV signal). If SHM_RDONLY is not specified, the memory can be both read and modified.

Detaching a shared memory segment is not the same as deleting it. Deletion is performed using the shmctl() IPC_RMID operation.

A child created by fork() inherits its parent's attached shared memory segments. Thus, shared memory provides an easy method of IPC between parent and child. During an exec(), all attached shared memory segments are detached. Shared memory segments are also automatically detached on process termination.

```
#include<stdio.h>
#include<sys/ipc.h>
#include<sys/shm.h>
```

```
main(int argc,char **argv)
{
int id;
char *p;

id=shmget(atoi(argv[1]),atoi(argv[2]),IPC_CREAT|0666);
perror("shmget");

p=shmat(id,0,0);
perror("shmat");
printf("P=%u\n",p);

printf("Enter data\n");
scanf("%s",p);

shmdt(p);
perror("shmdt");
}
```

The data written in shared memory can be read by any other process, which attaches the shared memory to itself. Following program shows process accessing data from shared memory.

```
#include<stdio.h>
#include<sys/ipc.h>
#include<sys/shm.h>

main(int argc,char **argv)
{
int id;
char *p;

id=shmget(atoi(argv[1]),atoi(argv[2]),IPC_CREAT|0666);
perror("shmget");

p=shmat(id,0,0);
printf("P=%u\n",p);

printf("data:%s\n",p);
}
```

If you follow the recommended approach of allowing the kernel to choose where to attach a shared memory segment, then (on the x86-32 architecture) memory layout appears with the segment being attached in the unallocated space between the upwardly growing heap and the downwardly growing stack.

Shared Memory Control Operations

The shmctl() system call performs a range of control operations on the shared memory segment identified by shmid.

```
#include <sys/types.h> /* For portability */

#include <sys/shm.h>

int shmctl(int shmid, int cmd, struct shmid_ds *buf);

Returns 0 on success, or –1 on error
```

The cmd argument specifies the control operation to be performed. The buf argument is required by the IPC_STAT and IPC_SET operations (described below), and should be specified as NULL for the remaining operations. Following are the generic control operations

IPC_RMID: Mark the shared memory segment and its associated shmid_ds data structure for deletion.

IPC_STAT: Place a copy of the shmid_ds data structure associated with this shared memory segment in the buffer pointed to by buf.

IPC_SET: Update selected fields of the shmid_ds data structure associated with this shared memory segment using values in the buffer pointed to by buf.

Each shared memory segment has an associated shmid_ds data structure of the following form:

```
struct shmid_ds {
struct ipc_perm shm_perm; /* Ownership and permissions */
size_t shm_segsz; /* Size of segment in bytes */
time_t shm_atime; /* Time of last shmat() */
time_t shm_dtime; /* Time of last shmdt() */
time_t shm_ctime; /* Time of last change */
pid_t shm_cpid; /* PID of creator */
pid_t shm_lpid; /* PID of last shmat() / shmdt() */
shmatt_t shm_nattch; /* Number of currently attached processes */
};
```

Shared memory allows two or more processes to share the same pages of memory. No kernel intervention is required to exchange data via shared memory. Once a process has copied data into a shared memory segment; that data is immediately visible to other processes. Shared memory provides fast IPC, although this speed advantage is somewhat offset by the fact that normally you must use some type of synchronization technique, such as a semaphore, to synchronize access to the shared memory.

IPC Mechanism…The Semaphore

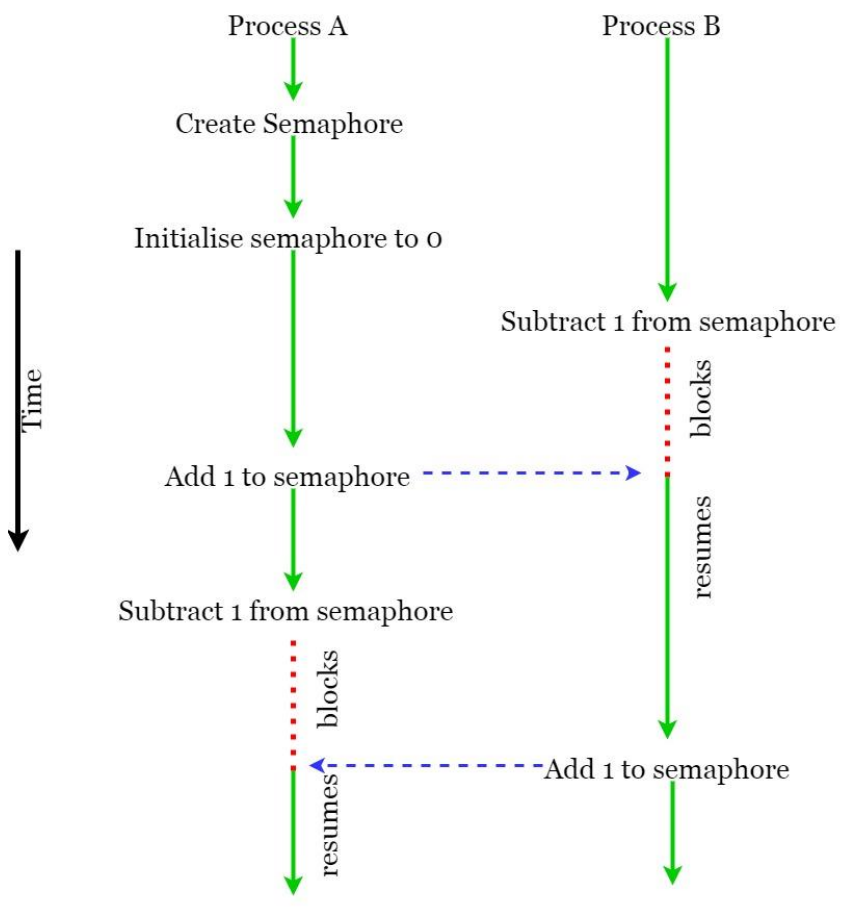
Unlike the IPC mechanisms described till now, System V semaphores are not used to transfer data between processes. Instead, they allow processes to synchronize their actions. One common use of a semaphore is to synchronize access to a block of shared memory, in order to prevent one process from accessing the shared memory at the same time as another process is updating it.

A semaphore is a kernel-maintained integer whose value is restricted to being greater than or equal to 0. Various operations (i.e., system calls) can be performed on a semaphore, including the following:

1) setting the semaphore to an absolute value;
2) adding a number to the current value of the semaphore;
3) subtracting a number from the current value of the semaphore; and
4) waiting for the semaphore value to be equal to 0.

The last two of these operations may cause the calling process to block. When lowering a semaphore value, the kernel blocks any attempt to decrease the value below 0. Similarly, waiting for a semaphore to equal 0 blocks the calling process if the semaphore value is not currently 0. In both cases, the calling process remains blocked until some other process alters the semaphore to a value that allows the operation to proceed, at which point the kernel wakes the blocked process.

Following diagram shows the use of a semaphore to synchronize the actions of two processes that alternately move the semaphore value between 0 and 1.



Using a semaphore to synchronize two processes

The general steps for using a System V semaphore are the following:

1) Create or open a semaphore set using semget().

2) Initialize the semaphores in the set using the semctl() SETVAL or SETALL operation. (Only one process should do this.)

3) Perform operations on semaphore values using semop(). The processes using the semaphore typically use these operations to indicate acquisition and release of a shared resource.

4) When all processes have finished using the semaphore set, remove the set using the semctl() IPC_RMID operation. (Only one process should do this.)

Creating or Opening a Semaphore Set

The semget() system call creates a new semaphore set or obtains the identifier of an

existing set.

```
#include <sys/types.h>
```

```
#include <sys/sem.h>

int semget(key_t key, int nsems, int semflg);

Returns semaphore set identifier on success, or –1 on error
```

If you are using semget() to create a new semaphore set, then nsems specifies the number of semaphores in that set, and must be greater than 0. If you are using semget() to obtain the identifier of an existing set, then nsems must be less than or equal to the size of the set (or the error EINVAL results). It is not possible to change the number of semaphores in an existing set.

The semflg argument is a bit mask specifying the permissions to be placed on a new semaphore set or checked against an existing set.In addition, zero or more of the following flags can be ORed (|) in semflg to control the operation of semget():

IPC_CREAT: If no semaphore set with the specified key exists, create a new set.

IPC_EXCL: If IPC_CREAT was also specified, and a semaphore set with the specified key already exists, fail with the error EEXIST.

On success, semget() returns the identifier for the new or existing semaphore set. Subsequent system calls referring to individual semaphores must specify both the semaphore set identifier and the number of the semaphore within that set. The semaphores within a set are numbered starting at 0.

Semaphore Control Operations

The semctl() system call performs a variety of control operations on a semaphore set

or on an individual semaphore within a set.

```
#include <sys/types.h>

#include <sys/sem.h>

int semctl(int semid, int semnum, int cmd, ... /* union semun arg */);

Returns nonnegative integer on success (see text); returns –1 on error
```

The semid argument is the identifier of the semaphore set on which the operation is to be performed. For those operations performed on a single semaphore, the semnum argument identifies a particular semaphore within the set. For other operations, this argument is ignored, and we can specify it as 0. The cmd argument specifies the operation to be performed.

The following generic control operations are the same ones that can be applied to other types of System V IPC objects. In each case, the semnum argument is ignored. These are IPC_RMID, IPC_STAT, IPC_SET.

Retrieving and initializing semaphore values

The following operations retrieve or initialize the value(s) of an individual semaphore or of all semaphores in a set. Retrieving a semaphore value requires read permission on the semaphore, while initializing the value requires alter (write) permission.

GETVAL: As its function result, semctl() returns the value of the semnum-th semaphore in the semaphore set specified by semid. The arg argument is not required.

SETVAL: The value of the semnum-th semaphore in the set referred to by semid is initialized to the value specified in arg.val.

GETALL: Retrieve the values of all of the semaphores in the set referred to by semid, placing them in the array pointed to by arg.array. The programmer must ensure that this array is of sufficient size.

SETALL: Initialize all semaphores in the set referred to by semid, using the values supplied in the array pointed to by arg.array. The semnum argument is ignored.

If another process is waiting to perform an operation on the semaphore(s) modified by the SETVAL or SETALL operations, and the change(s) made would permit that operation to proceed, then the kernel wakes up that process.

#include<stdio.h> *#include<sys/types.h>* *#include<sys/ipc.h>* *#include<sys/sem.h>* *main(int argc,char **argv)* *{* *int id,ret;* *if(argc!=2)* *{* *printf("Usage:./get semnum\n");* *return;* *}* *id=semget(4,5,IPC_CREAT|0666);*	*#include<stdio.h>* *#include<sys/types.h>* *#include<sys/ipc.h>* *#include<sys/sem.h>* *main(int argc,char **argv)* *{* *int id,ret;* *if(argc!=3)* *{* *printf("Usage:./set semnum semval\n");* *return;* *}* *id=semget(4,5,IPC_CREAT|0666);*

perror("semget"); *printf("semaphore id=%d\n",id);* *ret=semctl(id,atoi(argv[1]),GETVAL);* *perror("semctl");* *printf("semvalue ret=%d\n",ret);* *}* *$ cc semget.c –o get* *$ ./get 1* *semget: Success* *semaphore id=0* *semctl: Success* *semvalue ret=0* *$ ./get 1* *semget: Success* *semaphore id=0* *semctl: Success* *semvalue ret=5*	*perror("semget");* *printf("semaphore id=%d\n",id);* *ret=semctl(id,atoi(argv[1]),SETVAL,atoi(argv[2]));* *perror("semctl");* *printf("ret=%d\n",ret);* *}* *$cc semset.c –o set* *$ ./set 1 5* *semget: Success* *semaphore id=0* *semctl: Success* *ret=0*

Semaphore Operations

The semop() system call performs one or more operations on the semaphores in the semaphore set identified by semid.

```
#include <sys/types.h>

#include <sys/sem.h>

int semop(int semid, struct sembuf *sops, unsigned int nsops);

Returns 0 on success, or –1 on error
```

The sops argument is a pointer to an array that contains the operations to be performed, and nsops gives the size of this array (which must contain at least one element). The operations are performed atomically and in array order. The elements of the sops array are structures of the following form:

```
struct sembuf {
```

```
unsigned short sem_num; /* Semaphore number */
short sem_op; /* Operation to be performed */
short sem_flg; /* Operation flags (IPC_NOWAIT and SEM_UNDO) */
};
```

The sem_num field identifies the semaphore within the set upon which the operation is to be performed. The sem_op field specifies the operation to be performed:

1) If sem_op is greater than 0, the value of sem_op is added to the semaphore value. As a result, other processes waiting to decrease the semaphore value may be awakened and perform their operations. The calling process must have alter (write) permission on the semaphore.

2) If sem_op equals 0, the value of the semaphore is checked to see whether it currently equals 0. If it does, the operation completes immediately; otherwise, semop() blocks until the semaphore value becomes 0. The calling process must have read permission on the semaphore.

3) If sem_op is less than 0, decrease the value of the semaphore by the amount specified in sem_op. If the current value of the semaphore is greater than or equal to the absolute value of sem_op, the operation completes immediately. Otherwise, semop() blocks until the semaphore value has been increased to a level that permits the operation to be performed without resulting in a negative value. The calling process must have alter permission on the semaphore.

Semantically, increasing the value of a semaphore corresponds to making a resource available so that others can use it, while decreasing the value of a semaphore corresponds to reserving a resource for (exclusive) use by this process. When decreasing the value of a semaphore, the operation is blocked if the semaphore value is too low—that is, if some other process has already reserved the resource. When a semop() call blocks, the process remains blocked until one of the following occurs:

1) Another process modifies the value of the semaphore such that the requested operation can proceed.

2) A signal interrupts the semop() call.

3) Another process deletes the semaphore referred to by semid. In this case, semop() fails with the error EIDRM.

While it is usual to operate on a single semaphore at a time, it is possible to make a semop() call that performs operations on multiple semaphores in a set. The key point to note is that this group of operations is performed atomically; that is, semop() either performs all of the operations

immediately, if possible, or blocks until it would be possible to perform all of the operations simultaneously.

The following three processes are writing to same file simultaneously. The w1 process writing lowercase alphabets, w2 process writes uppercase alphabets and w3 process writes numeric characters. The semaphore is used to synchronize these processes so that after completion of lowercase letters, uppercase letters will be written and at last numbers will be appended to the data file.

W1.c	*W2.c*	*W3.c*
#include<stdio.h> *#include<sys/types.h>* *#include<sys/ipc.h>* *#include<sys/sem.h>* *#include<fcntl.h>* *main()* *{* *struct sembuf v;* *char a[]="abcdefghij klmnopqr";* *int i,id,fd;* *fd=open("data", O_RDWR| O_CREAT| O_APPEND, 0666);* *id=semget(4,5, IPC_CREAT| 0666);* *perror("semget");* *v.sem_num=1;* *v.sem_op=0;* *v.sem_flg=SEM_UNDO;* *printf("Before\n");* *semop(id,&v,1);* *printf("After\n");* *for(i=0;a[i];i++)* *{* *write(fd,&a[i],1);*	*#include<stdio.h>* *#include<sys/types.h>* *#include<sys/ipc.h>* *#include<sys/sem.h>* *#include<fcntl.h>* *main()* *{* *struct sembuf v;* *char a[]="ABCDEFGHIJ KLMNOPQR";* *int i,id,fd;* *fd=open("data", O_RDWR| O_CREAT| O_APPEND, 0666);* *id=semget(4,5, IPC_CREAT| 0666);* *perror("semget");* *v.sem_num=1;* *v.sem_op=-2;* *v.sem_flg=SEM_UNDO;* *printf("Before\n");* *semop(id,&v,1);* *printf("After\n");* *for(i=0;a[i];i++)* *{* *write(fd,&a[i],1);*	*#include<stdio.h>* *#include<sys/types.h>* *#include<sys/ipc.h>* *#include<sys/sem.h>* *#include<fcntl.h>* *main()* *{* *struct sembuf v;* *char a[]="1234567890";* *int i,id,fd;* *fd=open("data", O_RDWR| O_CREAT| O_APPEND, 0666);* *id=semget(4,5, IPC_CREAT| 0666);* *perror("semget");* *v.sem_num=1;* *v.sem_op=-1;* *v.sem_flg=SEM_UNDO;* *printf("Before\n");* *semop(id,&v,1);* *printf("After\n");* *for(i=0;a[i];i++)* *{* *write(fd,&a[i],1);*

sleep(1); *}* *printf("Done\n");* *v.sem_num=1;* *v.sem_op=2;* *v.sem_flg=SEM_UNDO;* *printf("Before\n");* *semop(id,&v,1);* *printf("After\n");* *}*	*sleep(1);* *}* *printf("Done\n");* *v.sem_num=1;* *v.sem_op=2;* *v.sem_flg=SEM_UNDO;* *printf("Before\n");* *semop(id,&v,1);* *printf("After\n");* *}*	*sleep(1);* *}* *printf("Done\n");* *v.sem_num=1;* *v.sem_op=1;* *v.sem_flg=SEM_UNDO;* *printf("Before\n");* *semop(id,&v,1);* *printf("After\n");* *}*

Disadvantages of System V Semaphores

System V semaphores have many of the same disadvantages as message queues including the following:

1) Semaphores are referred to by identifiers, rather than the file descriptors used by most other UNIX I/O and IPC mechanisms. This makes it difficult to perform operations such as simultaneously waiting both on a semaphore and on input from a file descriptor.

2) The use of keys, rather than filenames, to identify semaphores results in additional programming complexity.

3) The use of separate system calls for creating and initializing semaphores means that, in some cases, you must do extra programming work to avoid race conditions when initializing a semaphore.

4) The kernel doesn't maintain a count of the number of processes referring to a semaphore set. This complicates the decision about when it is appropriate to delete a semaphore set and makes it difficult to ensure that an unused set is deleted.

5) The programming interface provided by System V is overly complex. In the common case, a program operates on a single semaphore. The ability to simultaneously operate on multiple semaphores in a set is unnecessary.

However, unlike the situation with message queues, there are fewer alternatives to System V semaphores, and consequently there are more situations in which you may choose to employ them. One alternative to the use of semaphores is record locking.

Record Lock

A frequent application requirement is to read data from a file, make some change to that data, and then write it back to the file. As long as just one process at a time ever uses a file in this way,

then there are no problems. However, problems can arise if multiple processes are simultaneously updating a file.

Although you could use (say) semaphores to perform the required synchronization, using file locks is usually preferable, because the kernel automatically associates locks with files. You can use fcntl() to place locks on particular region of files.

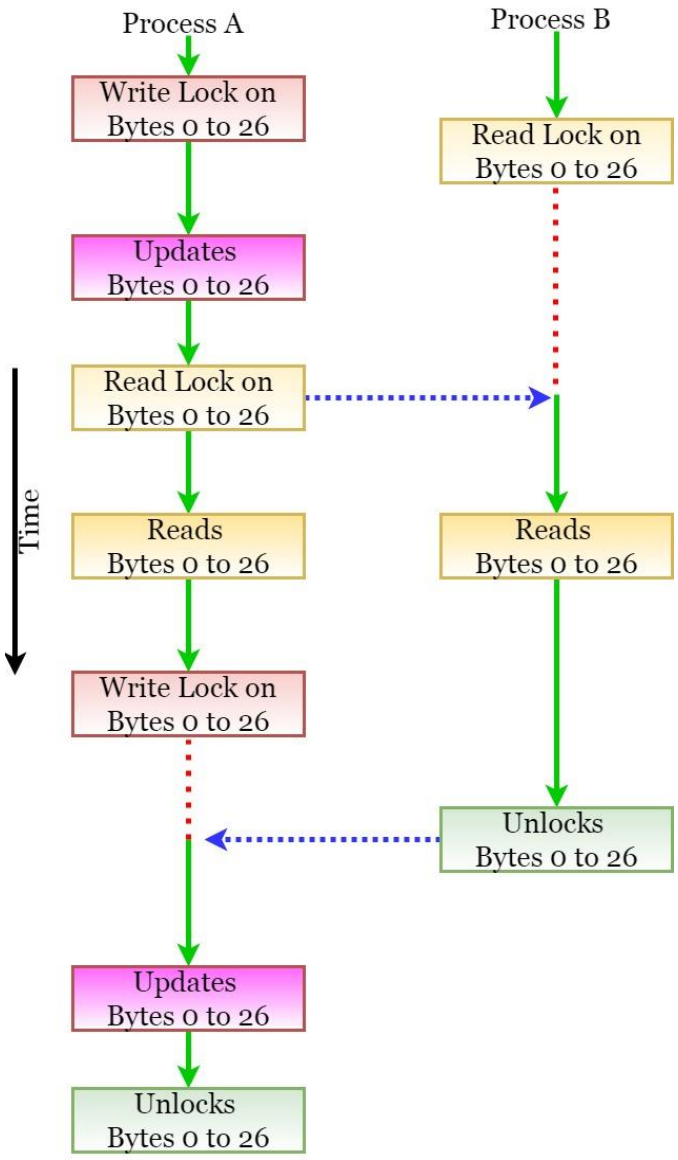
The general method of using fcntl() is as follows:

1) Place a lock on the file.
2) Perform file I/O.
3) Unlock the file so that another process can lock it.

Record Locking with fcntl()

Using fcntl(), you can place a lock on any part of a file, ranging from a single byte to the entire file. This form of file locking is usually called record locking. However, this term is a misnomer, because files on the UNIX system are byte sequences, with no concept of record boundaries.

Typically, fcntl() is used to lock byte ranges corresponding to the application defined record boundaries within the file; hence the origin of the term record locking. The terms byte range, file region, and file segment are less commonly used, but more accurate, descriptions of this type of lock. Following figure shows how record locking might be used to synchronize access by two processes to the same region of a file.



Using record locks to synchronize access to the same region of a file

The general form of the fcntl() call used to create or remove a file lock is as follows:

```
struct flock v;

/* Set fields of 'v' to describe lock to be placed or removed */

fcntl(fd, cmd, &v);
```

The fd argument is an open file descriptor referring to the file on which you wish to place a lock.

The flock structure

The flock structure defines the lock that you wish to acquire or remove. It is defined as follows:

```
struct flock {
short l_type; /* Lock type: F_RDLCK, F_WRLCK, F_UNLCK */
short l_whence; /* How to interpret 'l_start': SEEK_SET,SEEK_CUR, SEEK_END */
off_t l_start; /* Offset where the lock begins */
off_t l_len; /* Number of bytes to lock; 0 means "until EOF" */
pid_t l_pid; /* Process preventing our lock (F_GETLK only) */
};
```

The l_type field indicates the type of lock you want to place. It is specified as one of the values given below.

Lock type	Description
F_RDLCK	Place a read lock
F_WRLCK	Place a write lock
F_UNLCK	Remove an existing lock

Any number of processes can hold read locks on a file region, but only one process can hold a write lock, and that lock excludes read and write locks by other processes.

In order to place a read lock on a file, the file must be open for reading. Similarly, to place a write lock, the file must be open for writing. To place both types of locks, you open the file read-write (O_RDWR). Attempting to place a lock that is incompatible with the file access mode results in the error EBADF.

The l_whence, l_start, and l_len fields together specify the range of bytes to be locked. The first two of these fields are analogous to the whence and offset arguments to lseek(). The l_start field specifies an offset within the file that is interpreted with respect to one of the following:

1) the start of the file, if l_whence is SEEK_SET;

2) the current file offset, if l_whence is SEEK_CUR; or

3) the end of the file, if l_whence is SEEK_END.

In the last two cases, l_start can be a negative number, as long as the resulting file position doesn’t lie before the start of the file (byte 0).The l_len field contains an integer specifying the number of bytes to lock, starting from the position defined by l_whence and l_start. It is possible to lock nonexistent bytes past the end of the file, but it is not possible to lock bytes before the start of the file. Specifying 0 in l_len has the special meaning “lock all bytes from the point

specified by l_start and l_whence through to the end of the file, no matter how large the file grows." To lock the entire file, you can specify l_whence as SEEK_SET and both l_start and l_len as 0.

The cmd argument

When working with file locks, three possible values may be specified for the cmd argument of fcntl(). The first two are used for acquiring and releasing locks:

F_SETLK

Acquire (l_type is F_RDLCK or F_WRLCK) or release (l_type is F_UNLCK) a lock on the bytes specified by v. If an incompatible lock is held by another process on any part of the region to be locked, fcntl() fails with the error EAGAIN.

F_SETLKW: This is the same as F_SETLK, except that if another process holds an incompatible lock on any part of the region to be locked, then the call blocks until the lock can be granted. If you are handling signals and have not specified SA_RESTART, then an F_SETLKW operation may be interrupted (i.e., fail with the error EINTR). You can take advantage of this behavior to use alarm() to set a timeout on the lock request.

The remaining fcntl() operation is used to determine whether you can place a lock on a given region:

F_GETLK: Check if it would be possible to acquire the lock specified in flockstr, but don't actually acquire it. The l_type field must be F_RDLCK or F_WRLCK.If the lock would be permitted (i.e., no incompatible locks exist on the specified file region), then F_UNLCK is returned in the l_type field, and the remaining fields are left unchanged. If one or more incompatible locks exist on the region, then flockstr returns information about one of those locks (it is indeterminate which), including its type (l_type), range of bytes (l_start and l_len; l_whence is always returned as SEEK_SET), and the process ID of the process holding the lock (l_pid).

There are two processes writing in a file, one process write lower case letters where as other process writes uppercase letters. Record lock is used to ensure that till one process writes, other should not interrupt it. Check the following code.

lowrcsltr.c	uprcsltr.c
#include<stdio.h> *#include<unistd.h>* *#include<fcntl.h>* *main()* *{* *struct flock v;*	*#include<stdio.h>* *#include<unistd.h>* *#include<fcntl.h>* *main()* *{* *struct flock v;*

char a[]="abcdefghijklmnopqr"; *int i,fd;* *fd=open("data",O_RDWR |O_CREAT |O_APPEND,0666);* *v.l_type=F_WRLCK;* *v.l_whence=SEEK_SET;* *v.l_start=0;* *v.l_len=0;* *printf("Before\n");* *fcntl(fd,F_SETLKW,&v);* *printf("After\n");* *for(i=0;a[i];i++)* *{* *write(fd,&a[i],1);* *sleep(1);* *}* *v.l_type=F_UNLCK;* *fcntl(fd,F_SETLKW,&v);* *printf("Done\n");* *}*	*char a[]="ABCDEFGHIJKLMNOPQR";* *int i,fd;* *fd=open("data",O_RDWR |O_CREAT |O_APPEND,0666);* *v.l_type=F_WRLCK;* *v.l_whence=SEEK_SET;* *v.l_start=0;* *v.l_len=0;* *printf("Before\n");* *fcntl(fd,F_SETLKW,&v);* *printf("After\n");* *for(i=0;a[i];i++)* *{* *write(fd,&a[i],1);* *sleep(1);* *}* *v.l_type=F_UNLCK;* *fcntl(fd,F_SETLKW,&v);* *printf("Done\n");* *}*

Some Important Points

Note the following points regarding the acquisition and release of locks created with fcntl():

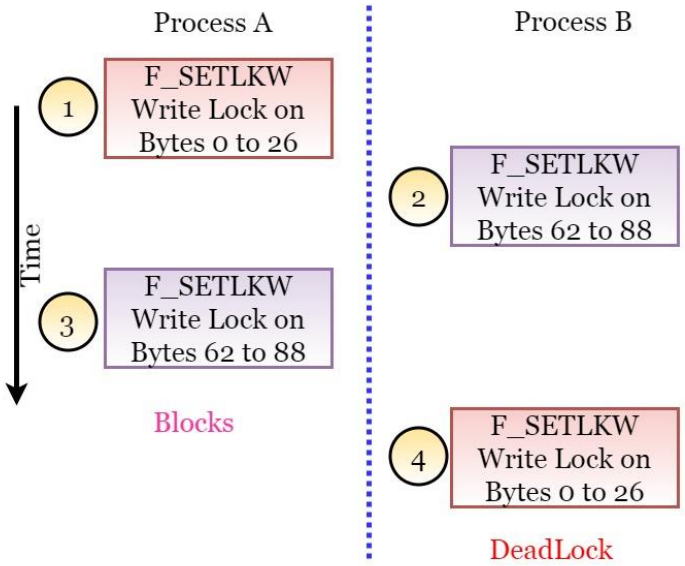
1) Unlocking a file region always immediately succeeds. It is not an error to unlock a region on which you don't currently hold a lock.

2) At any time, a process can hold just one type of lock on a particular region of a file. Placing a new lock on a region you have already locked either results in no change (if the lock type is the same as the existing lock) or atomically converts the existing lock to the new mode.

3) Placing a lock of a different mode in the middle of a lock you already hold results in three locks: two smaller locks in the previous mode are created on either side of the new lock.

Deadlock

When using F_SETLKW, you need to be aware of the type of scenario illustrated in following figure.



Deadlock when two processes deny each other's lock requests

In this scenario, each process's second lock request is blocked by a lock held by the other process. Such a scenario is referred to as a deadlock. If unchecked by the kernel, this would leave both processes blocked forever. To prevent this possibility, the kernel checks each new lock request made via F_SETLKW to see if it would result in a deadlock situation. If it would, then the kernel selects oneof the blocked processes and causes its fcntl() call to unblock and fail with the error EDEADLK.

Deadlock situations are detected even when placing locks on multiple different files, as are circular deadlocks involving multiple processes. By circular deadlock, you mean, for example, process A waiting to acquire a lock on a region locked by process B, process B waiting on a lock held by process C, and process C waiting on a lock held by process A.

Summarizing System V IPC Mechanisms

Pipes were the first method of IPC under the UNIX system, and they are used frequently by the shell, as well as in other applications. A pipe is a unidirectional, limited capacity byte stream that can be used for communication between related processes. Although blocks of data of any size can be written to a pipe, only writes that do not exceed PIPE_BUF bytes are guaranteed to be atomic. As well as being used as a method of IPC, pipes can also be used for process synchronization. When using pipes, you must be careful to close unused descriptors in order to ensure that reading processes detect end-of-file and writing processes receive the SIGPIPE signal or the EPIPE error.

FIFOs operate in exactly the same way as pipes, except that they are created using mkfifo(), have a name in the file system, and can be opened by any process with appropriate permissions. By default, opening a FIFO for reading blocks until another process opens the FIFO for writing, and vice versa.

The System V three IPC mechanisms are message queues, which allow processes to exchange messages; semaphores, which allow processes to synchronize access to shared resources; and shared memory, which allows two or more processes to share the same pages of memory.

These three IPC mechanisms have many similarities in their APIs and semantics. For each IPC mechanism, a get system call creates or opens an object. Given an integer key, the get calls return an integer identifier used to refer to the object in subsequent system calls. Each IPC mechanism also has a corresponding a ctl call that is used to delete an object and to retrieve and modify various attributes (e.g., ownership and permissions) in an object's associated data structure.

The ipcs command lists the System V IPC objects that currently exist on the system. The ipcrm command is used to remove System IPC objects.

System V message queues allow processes to communicate by exchanging messages consisting of a numeric type plus a body containing arbitrary data. The distinguishing features of message queues are that message boundaries are preserved and that the receiver(s) can select messages by type, rather than reading messages in first-in, first-out order.

Shared memory allows two or more processes to share the same pages of memory. No kernel intervention is required to exchange data via shared memory. Once a process has copied data into a shared memory segment; that data is immediately visible to other processes. Shared memory provides fast IPC, although this speed advantage is somewhat offset by the fact that normally you must use some type of synchronization technique, such as a System V semaphore, to synchronize access to the shared memory.

System V semaphores allow processes to synchronize their actions. This is useful when a process must gain exclusive access to some shared resource, such as a region of shared memory. Semaphores are created and operated upon in sets containing one or more semaphores. Each semaphore within a set is an integer whose value is always greater than or equal to 0. The semop() system call allows the caller to add an integer to a semaphore, subtract an integer from a semaphore, or wait for a semaphore to equal 0. The last two of these operations may cause the caller to block.

The fcntl() system call places locks ("record locks") on any region of a file, ranging from a single byte to the entire file. Two types of locks may be placed: read locks and write locks. If a blocking (F_SETLKW) lock request would bring about a deadlock situation, then the kernel causes fcntl() to fail (with the error EDEADLK) in one of the affected processes.